

NON-UNIFORM LEARNABILITY CONVEX LEARNING BIAS & VARIANCE

Dr. Krishna Bathula

Learning Objectives

1. Non-Uniform Learnability
2. Convex Learning
3. Gradient Descent
4. Bias & Variance Tradeoff
5. Underfitting & Overfitting
6. VC Dimension
7. Structural Risk Minimization
8. Generalized Linear Models

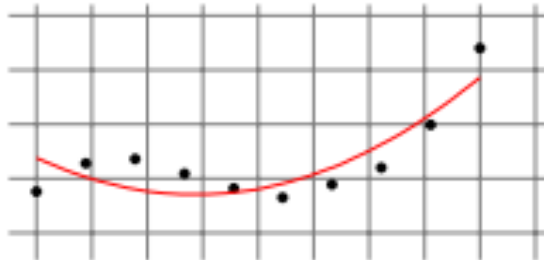
Non-Uniform Learnability

- When approaching a learning problem, a natural question is how many examples we need to collect in order to learn it.
- **PAC** learning gives an exact answer.
- However, for both **nonuniform learning** and **consistency**, we do not know in advance how many examples are required to learn H .
- In nonuniform learning this number depends on the **best hypothesis in H** and the consistency depends on the **underlying distribution**.
- If the **estimation(training) error** of the predictor we learn is small, **its risk may still be large** if H has a large **approximation (test) error**.

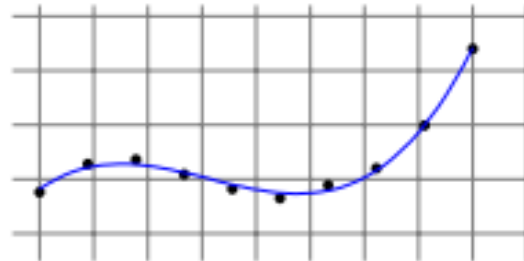
Non-Uniform Learnability

- When a nonuniform algorithm fails, we can consider a **different weighing function** over (subsets of) hypotheses.
- In Polynomial Regression, the learner adapts to the best fit, not with a straight line but a **curved line** based on non-uniform learnability with the **feature scaling**.

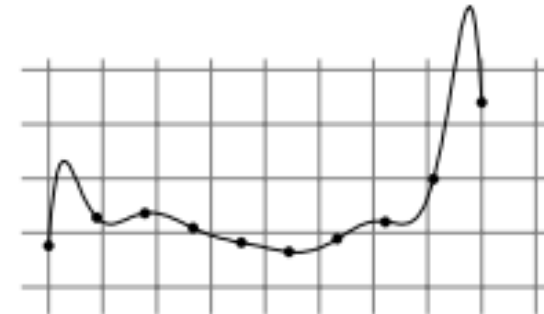
degree 2



degree 3



degree 10



- **Nonuniform learning** enables learning many concept classes not learnable by the previously discussed definitions.

Convex Learning

Convex learning refers to the use of **convex optimization** techniques to solve machine learning problems.

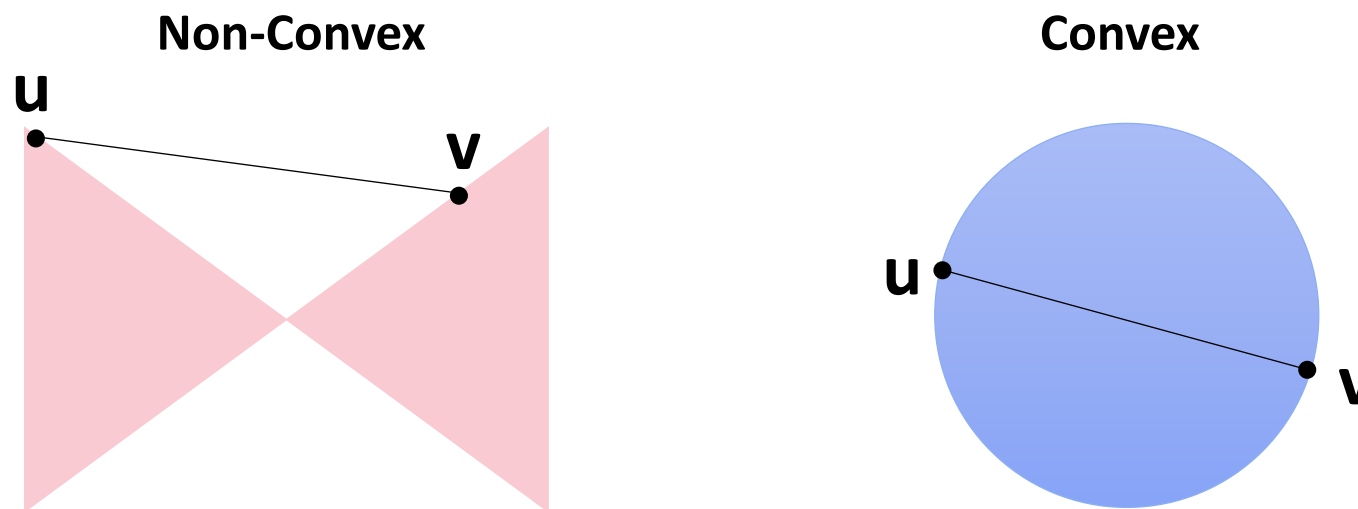
A convex learning problem is a problem whose **hypothesis class is a convex set**, and whose loss function is a convex function.

Convex learning algorithms can be used for a wide range of tasks, including regression, classification, clustering, and feature selection.

These algorithms can be applied to both supervised and unsupervised learning tasks.

Convex optimization problems have certain properties that make them easier to solve and ensure that the solutions obtained are **global optima**.

Convex Learning



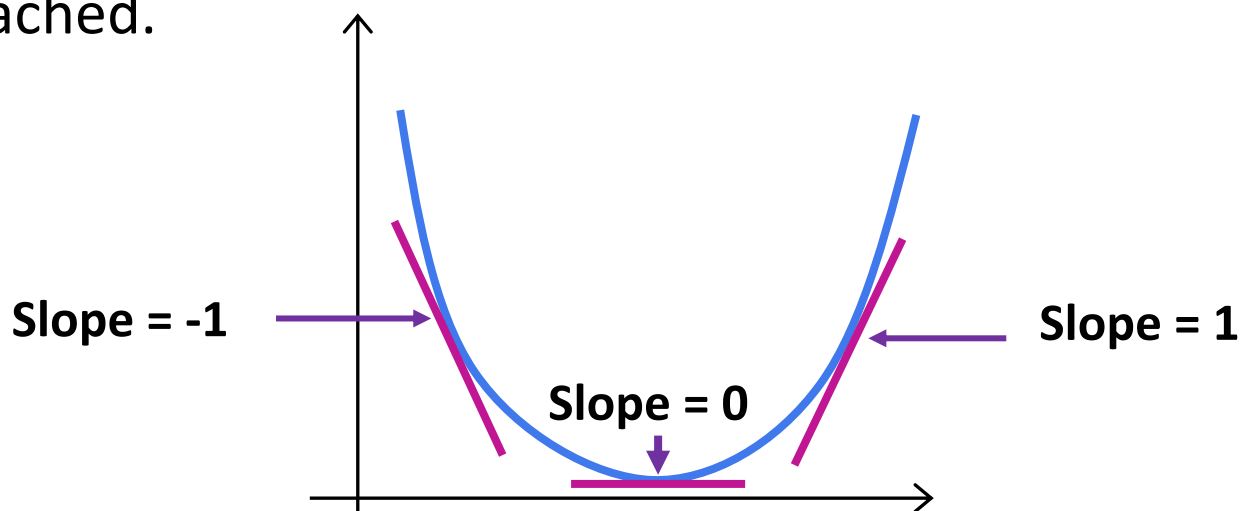
- In the **non-convex** set consider the two points, **u** and **v**. A line drawn between the two points is not contained in the set. Hence, it cannot be said to be convex.
- However, with a **convex set** no matter where we place our two points in the circular set, the line **will always be contained inside** it. Hence, it is said to be a convex set.

Optimization of a Function

A function at an **inflection point** can be increasing or decreasing. This is determined by the first derivative of the function.

If the **slope = -1** or the derivative of the function is **negative**, it is decreasing.
Otherwise, it is **positive** i.e., **slope = 1**

Whenever first derivative is equal to zero, that is the slope is 0, the local minimum is reached.

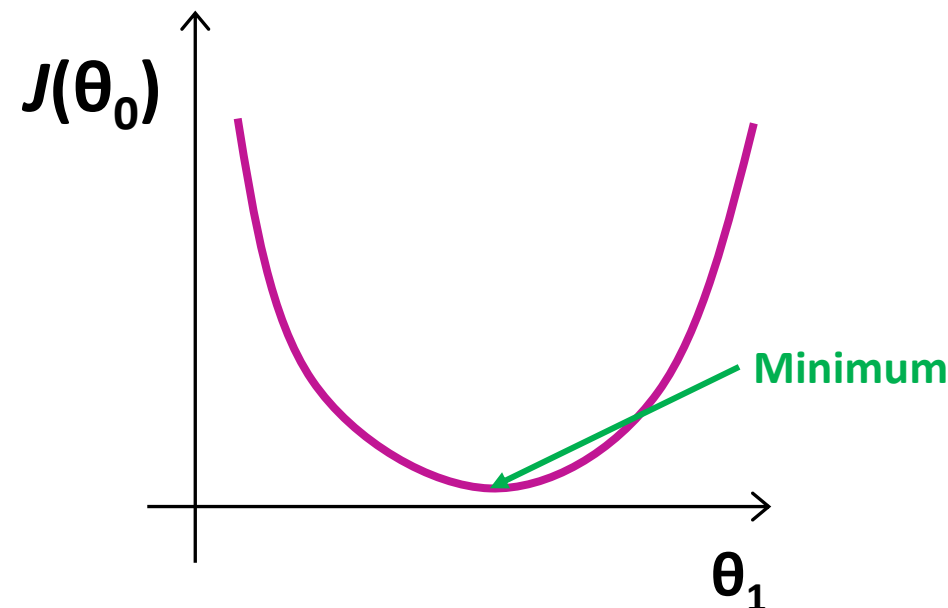


- Instantaneous rate of change of a function with respect to one of its variables
- Slope of the tangent line at a given point

Optimizing Cost Function

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1



Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h(x^i) - y^i)^2$

Goal: **Minimize** $J(\theta_0, \theta_1)$
 θ_0, θ_1

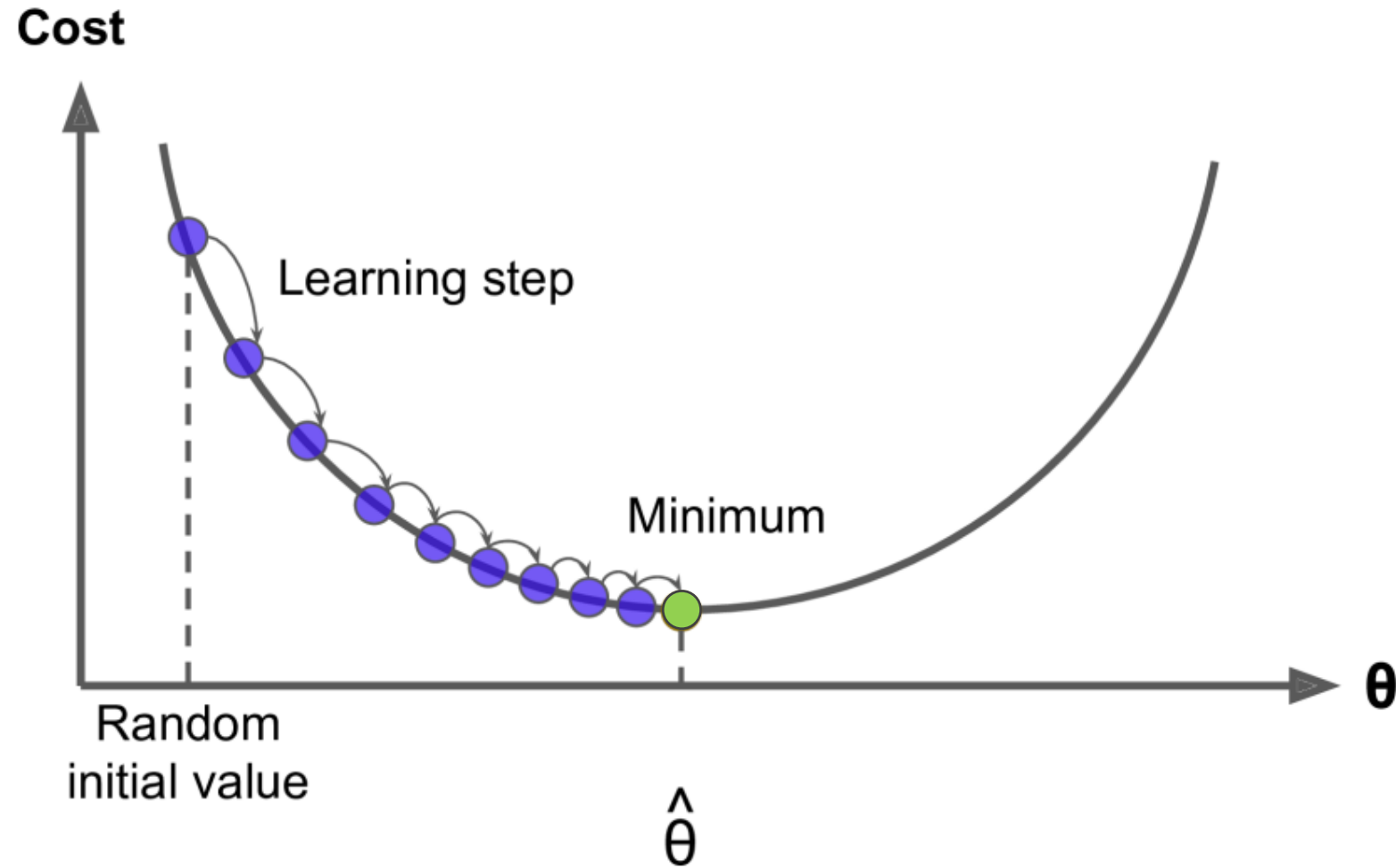
The cost function must be minimized to find the best parameter θ .

Gradient Descent

Gradient Descent is a generic optimization algorithm capable of finding **optimal solutions** to a wide range of problems.

- The general approach of Gradient Descent is to **tweak parameters iteratively** in order to **minimize a Cost Function**.
- **Analogy:**
 - Assume that you are lost in a dense fog on top of a mountain, and you can only feel the slope of the ground below your feet.
 - A good strategy to get to the bottom of the valley quickly is to go downhill in the direction of the steepest slope.
 - This exact approach is adapted by Gradient Descent.
- Gradient Descent measures the local gradient of the error function with regard to the parameter vector θ and it goes in the direction of descending gradient. Once the **gradient is 0**, the **minimum** is reached.

Gradient Descent



Gradient Descent Optimization

The optimization process of Gradient Descent:

1. First, the **parameters** are filled with **random variables**. This is called **Random Initialization**.
2. Then model parameters are tweaked step-wise to **minimize the Cost Function**.
3. The tweaking is repeated until the **algorithm converges to a minimum** i.e., the cost function reaches minimum.
4. An **important parameter** in Gradient Descent is the **size of the steps** taken to tweak the model parameters. This is determined by the **Learning Rate**

Gradient Descent - Convergence Algorithm

Repeat until Convergence

{

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

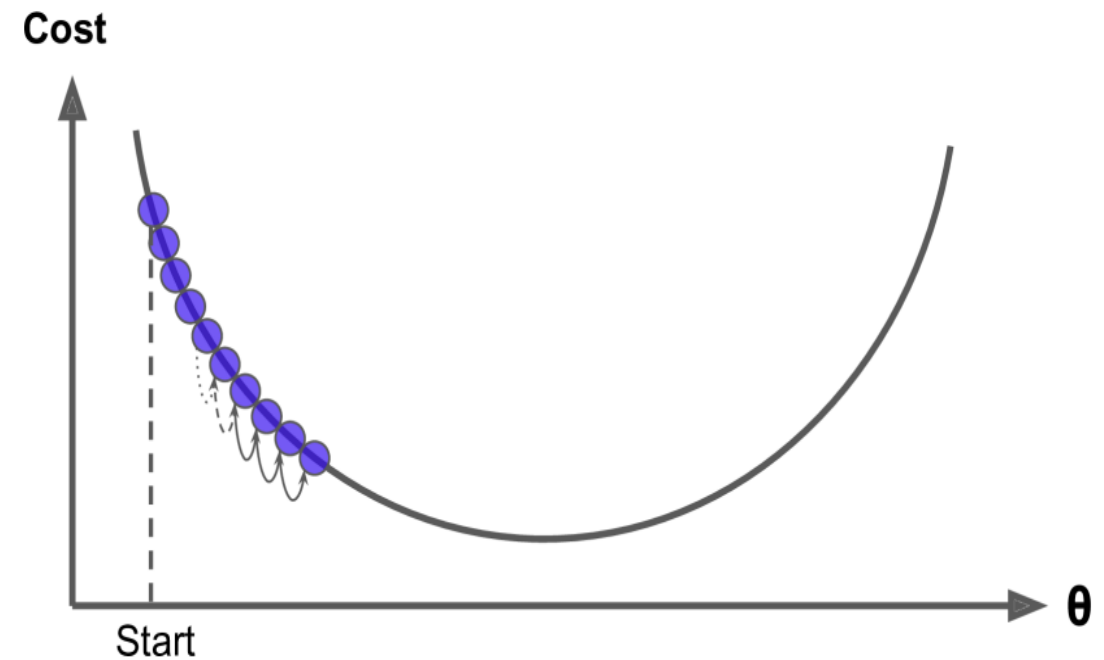
}

α - is the Learning Rate

Learning Rate

In machine learning and statistics, the **Learning Rate** is a tuning parameter in an optimization algorithm that determines the step size at each iteration while moving toward a minimum of a loss function.

If the **Learning Rate** is too **small**, then the algorithm will have to go through many iterations to converge, which will take longer time.

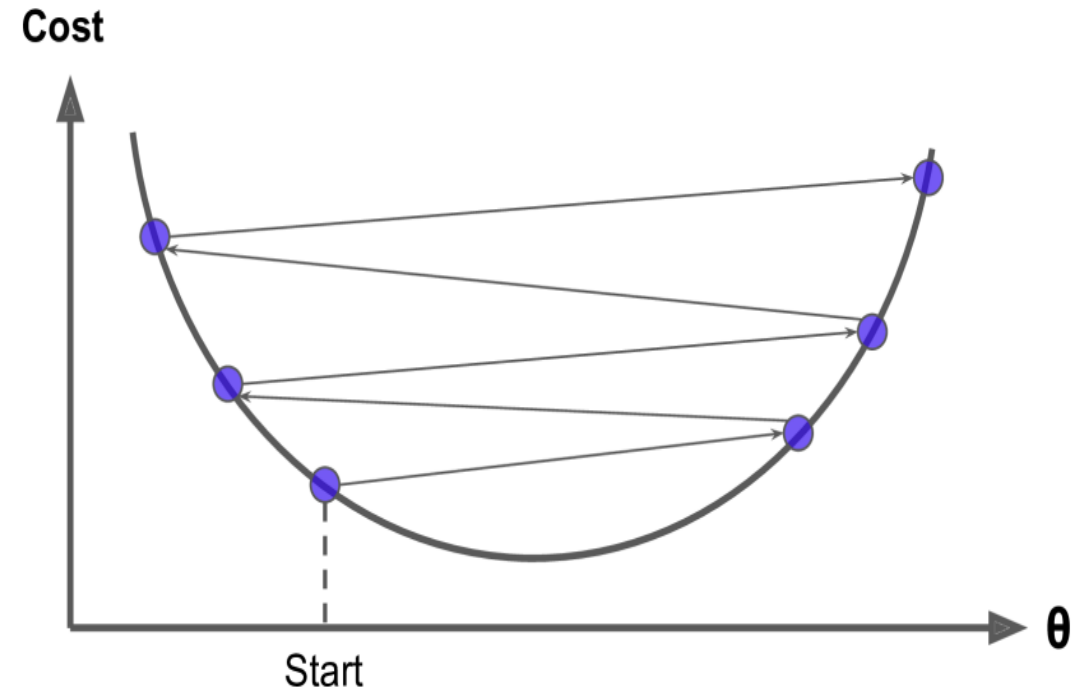


Learning Rate is too small

Learning Rate

On the other hand, if the **Learning Rate** is too **high**, then the step will jump across the valley and end up on the other side, possibly even higher up than before.

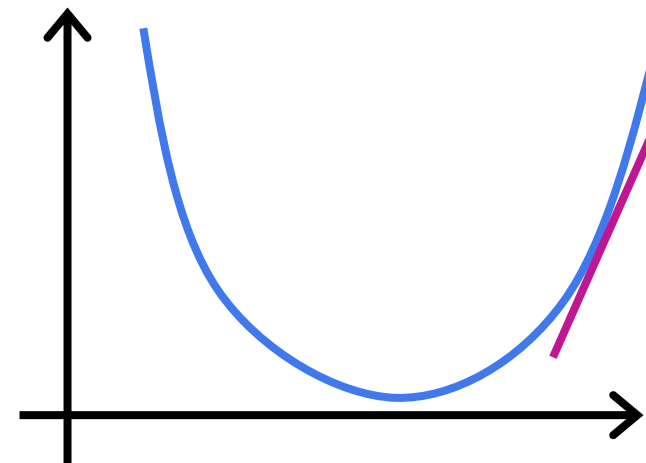
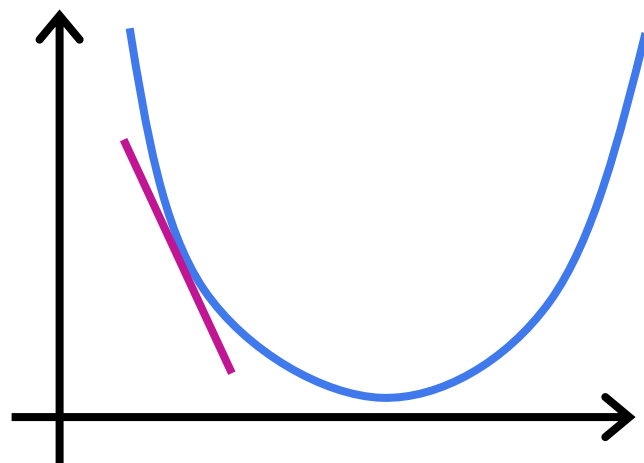
This might make the algorithm **diverge** with larger and larger values and larger values, failing to find a good solution.



Learning Rate is too Large

Gradient Descent Optimization

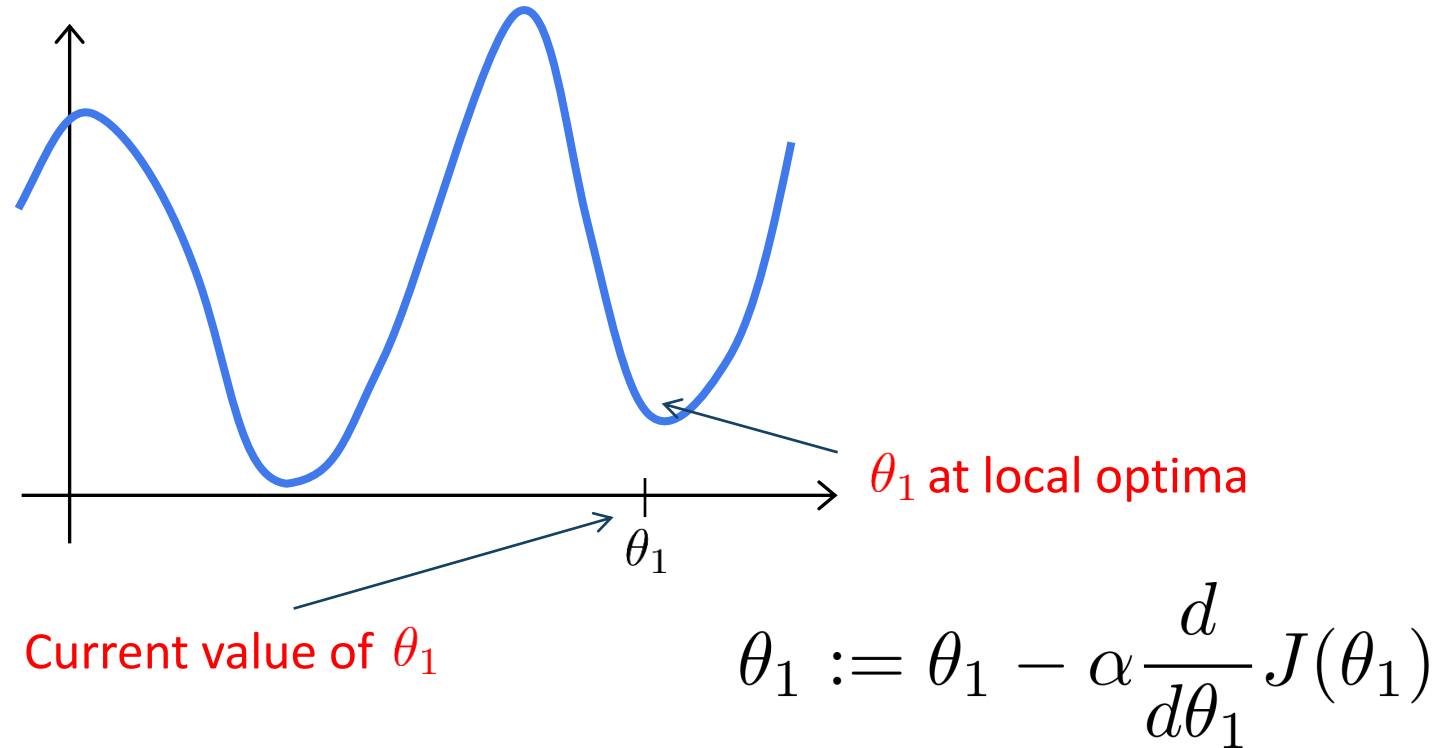
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$



Gradient descent can converge to a local minimum, even with the **learning rate α is fixed**. -- Adjust theta

As we approach a local minimum, gradient descent will automatically take smaller steps. So, no need to decrease α over time.

Gradient Descent Optimization



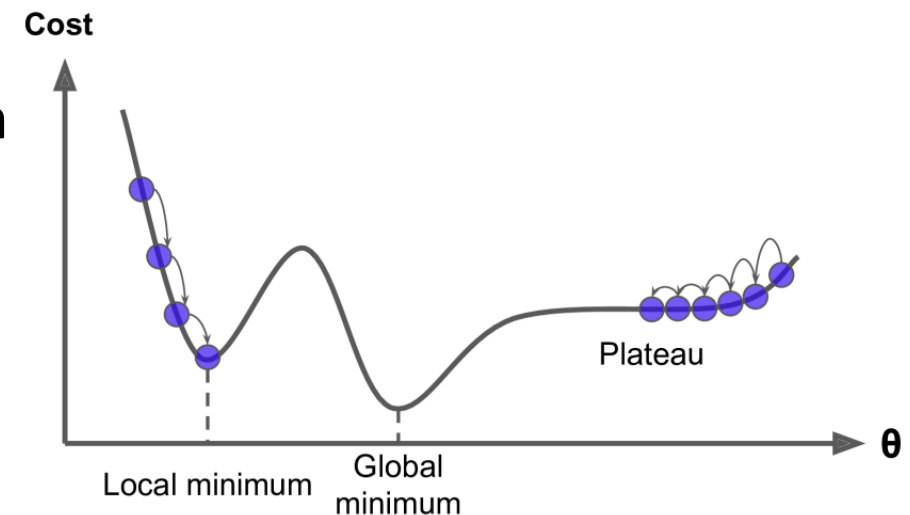
Gradient Descent - Pitfalls

Not all Cost Functions appear as smooth as a curve (regular bowls).

There may be **holes**, **ridges**, **plateaus**, and all sorts of **irregular terrains**, making **convergence** to the minimum very **difficult**.

The challenges with Gradient Descent are:

- If the random initialization starts the algorithm from the **left**, then it will converge to a **local minimum**, which is not as good as the global minimum.
- If it starts on the **right**, then it will take a very long time to cross the plateau and reach the global minimum. Stopping early will avoid.



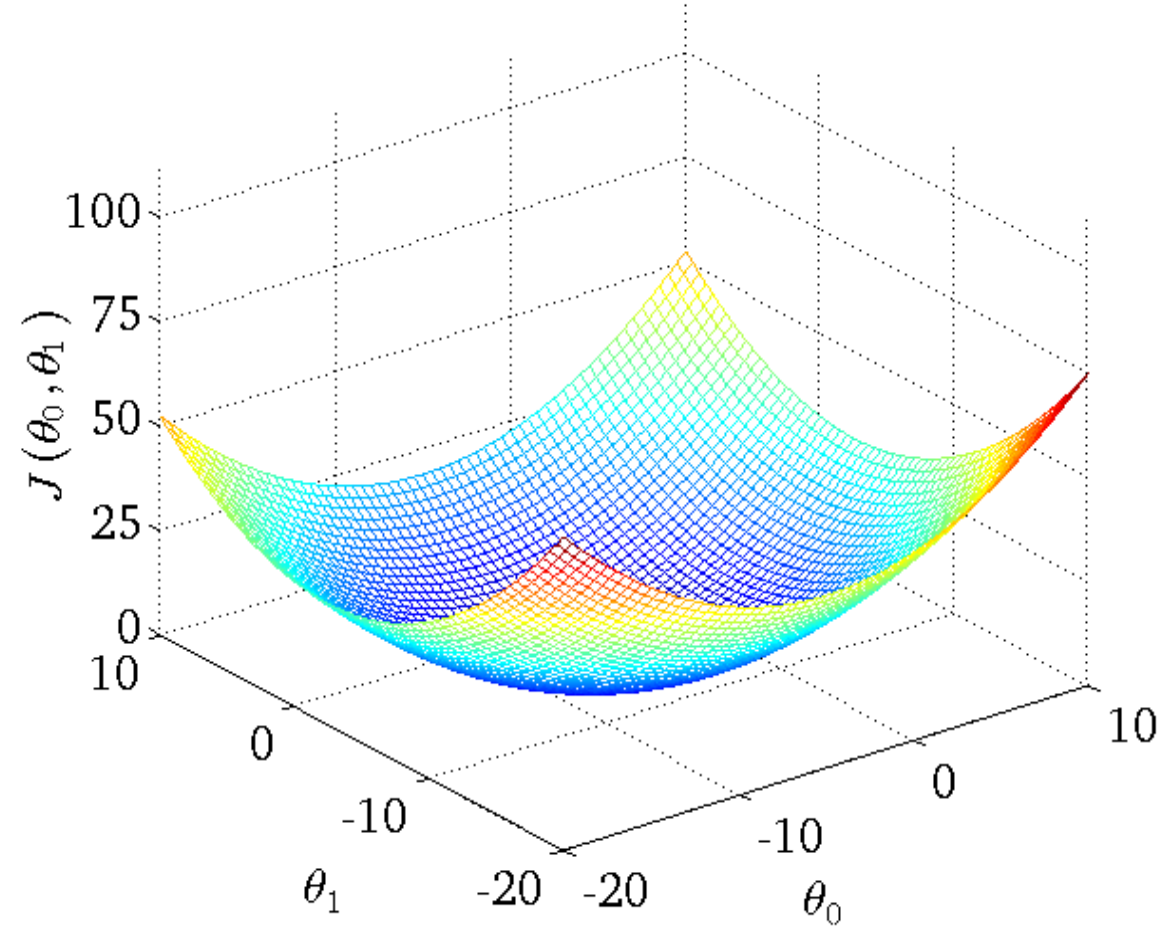
Gradient Descent- Convex Function

The **MSE cost function** for a **Linear Regression** model is a convex function.

This implies that there are **no local minima**, just **one global minimum**.

It is also a continuous function with a slope that never changes abruptly.

Here Gradient Descent is guaranteed to approach arbitrarily close to the global minimum



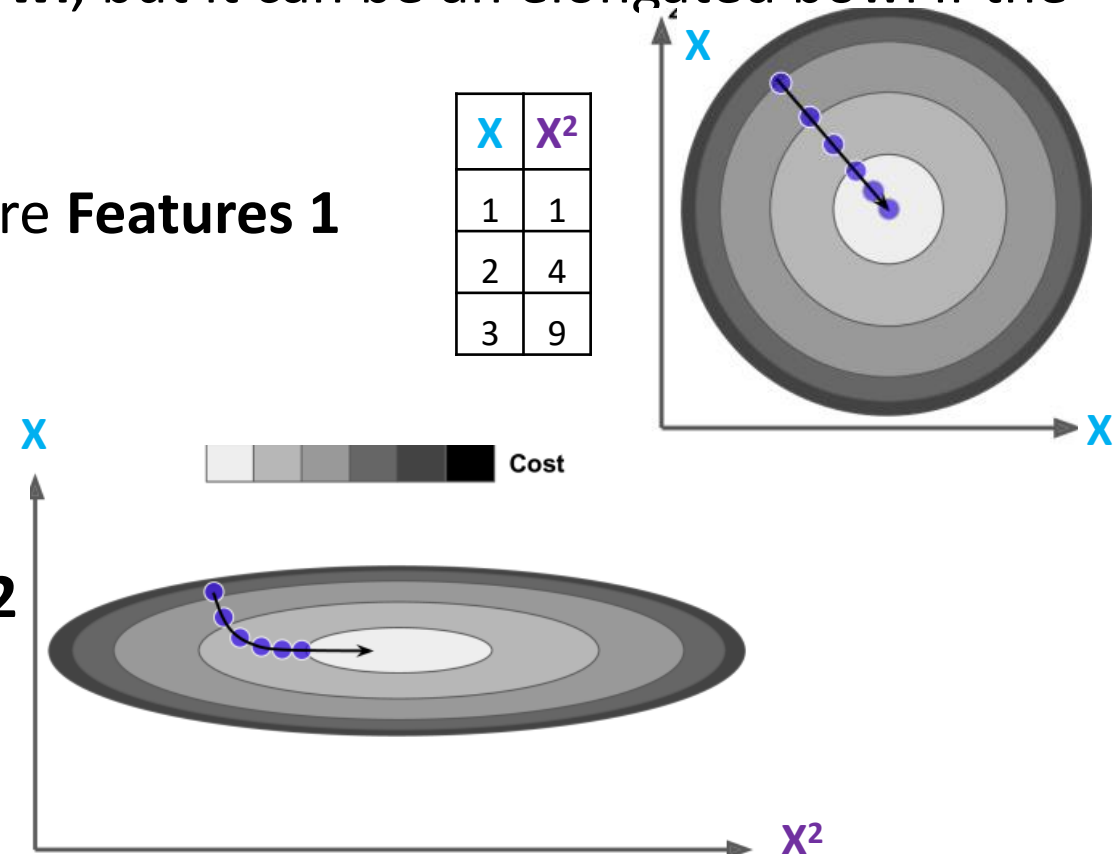
Gradient Descent – Feature Scaling

In **Polynomial Regression**, we had features that represent

The cost function has the shape of a **bowl**, but it can be an elongated bowl if the features have very different scales.

Gradient Descent on a training set where **Features 1 and Feature 2** have the **same scale**

GD on a training set where **Feature 1** has much smaller values than **Feature 2**



Multiple Linear Regression

Multiple linear regression is a statistical technique that uses **several explanatory variables** to predict the outcome of a response variable.

The goal of multiple linear regression is to model the **linear relationship between the explanatory** (independent) variables and response (dependent) variables.

Linear regression can only be used when one has two continuous variables—an independent variable and a dependent variable.

The multiple regression model is **based on the assumption** that there is a linear relationship between the dependent variables and the independent variables and the independent variables **are not too highly correlated** with each other.

Linear Regression

Consider another example of housing prices prediction. The independent variables are :

1. ($\mathbf{x}_1$) area of the house in square feet,
2. ($\mathbf{x}_2$) no. of bedrooms,
3. ($\mathbf{x}_3$) location

In general, when designing a learning problem, it will be up to you to decide what features to choose

$$\mathbf{h}_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}_0 + \boldsymbol{\theta}_1\mathbf{x}_1 + \boldsymbol{\theta}_2\mathbf{x}_2 + \boldsymbol{\theta}_3\mathbf{x}_3\ldots$$

Here, the $\boldsymbol{\theta}_i$'s are the parameters (also called coefficients or weights) parameterizing the space of linear functions mapping from X to Y . To simplify let us denote $\mathbf{h}_{\boldsymbol{\theta}}(\mathbf{x})$ as $\mathbf{h}(\mathbf{x})$

Bias

- The generalization error(training error + test error) is due to **wrong assumptions**, such as assuming that the **data is linear when it is quadratic**.
- The training data with a **high error rate** has a **high Bias** value.
- When the **error rate is low** it is a **low bias** value.
- A high-bias model is most likely to **underfit** the training data.
- The Empirical Risk Minimization reduces the loss on the training set.
- An important theoretical result of statistics and Machine Learning is that a model's total error can be expressed as the sum of three very different errors:
1. Bias 2. Variance 3. Irreducible error

Variance

Variance refers to the **amount by which $\hat{y}$ would change** if we estimated it using a **different training data set**. More flexible statistical methods have higher variance.

Explanation: Different training data sets will result in a different $\hat{y}$. But ideally, the estimate for y should not vary too much between training sets.

However, if a method has high variance then small changes in the training data can result in large changes in $\hat{y}$.

- This is due to the model's excessive sensitivity to small variations in the training data.
- The difference between the **error rate of training data** and **test data** is called **variance**.
- If the difference **is high** then it's called **high variance** and when the **difference of errors is low** then it's called **low variance**.
- A model with many **degrees of freedom** (such as a high-degree polynomial model where x is raised to the power of 10) is likely to have high variance, and thus to overfit the training data.

Bias – Variance Trade-off

Irreducible error

- This part of error is due to the noisiness of the data itself.
- The only way to reduce this part of the error is to **clean up the data** (e.g., fix the data sources, remove data inconsistencies, or detect and remove outliers).

Increasing a model's complexity will typically **increase its variance** and **reduce its bias**.

Conversely, **reducing a model's complexity** **increases its bias** and **reduces its variance**.

This is why it is called a **tradeoff**.

Underfitting

- In supervised learning, **underfitting** happens when a model unable to capture the underlying pattern of the data.
- These models usually have **high bias** and **low variance**.
- It happens when we have **very less amount of data** to build an accurate model or when we try to build a **linear model with nonlinear data**.
- Also, these kind of models are very simple to capture the **complex patterns** in data like Linear and logistic regression.
- Model **accuracy is bad** with both the training and test data.
- This condition is referred to as the case with **high bias** for training data and having **high variance** for test data.

Overfitting

In supervised learning, **overfitting** happens when our model captures the noise along with the underlying pattern in data.

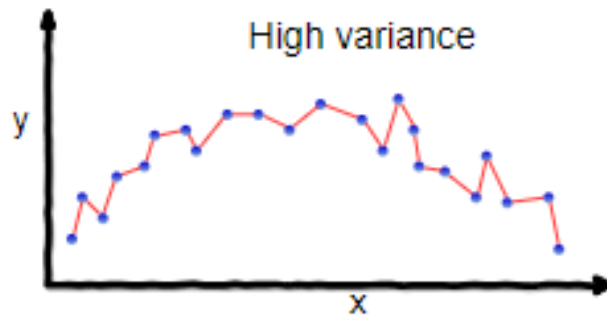
The **model performs well** with **training data** but **fails to perform well** with **test data**.

It happens when we train our model a lot over **noisy datasets**.

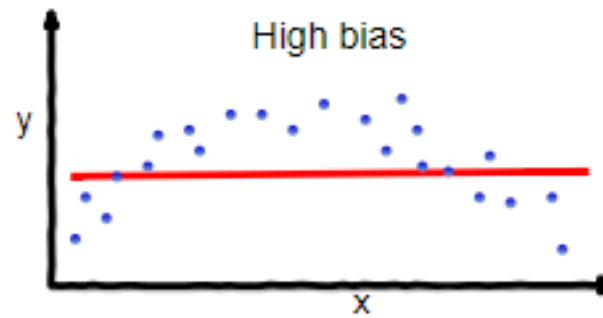
These models have **low bias and high variance**.

These models are very complex like Decision trees which are prone to overfitting.

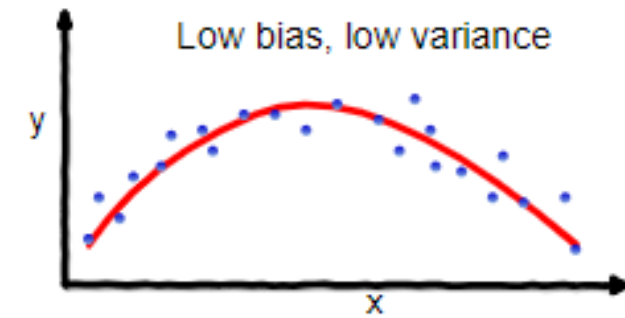
Bias Variance Tradeoff



overfitting

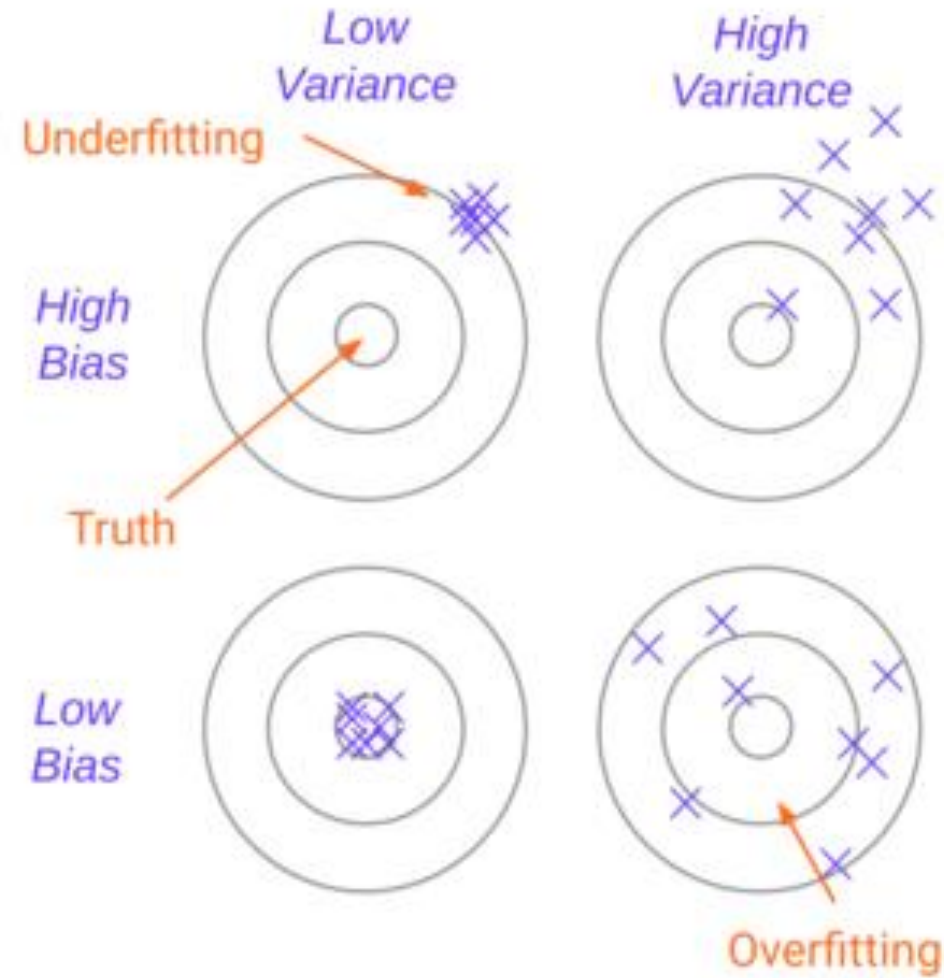


underfitting



Good balance

Bias – Variance Trade-off



Bias Variance Tradeoff

1. If our model is **too simple** and has very few parameters then it may **have high bias and low variance**.
2. On the other hand, if our model has a **large number of parameters** then it's going to have **high variance and low bias**.
3. So, we need to find the right/good balance without overfitting and underfitting the data.
4. This tradeoff in complexity is why there is a tradeoff between bias and variance. **A model can't be more complex and less complex at the same time.**

Total Error

To build a good model, we need to find a good balance between bias and variance such that it minimizes the total error.

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Shattering

- In machine learning and statistical learning theory, "**shattering**" refers to the **ability of a hypothesis class to correctly classify** all possible labels of a set of data points.
- The set of data points is divided into two classes (e.g., positive and negative classes in binary classification). Labeling refers to assigning a binary label to each data point.
- A hypothesis class is said to "shatter" a set of data points if, for every possible way of labeling those points, **there exists at least one function in the hypothesis class that correctly classifies the data** according to that labeling.

Shattering

The hypothesis class can represent any possible label/class of the data.

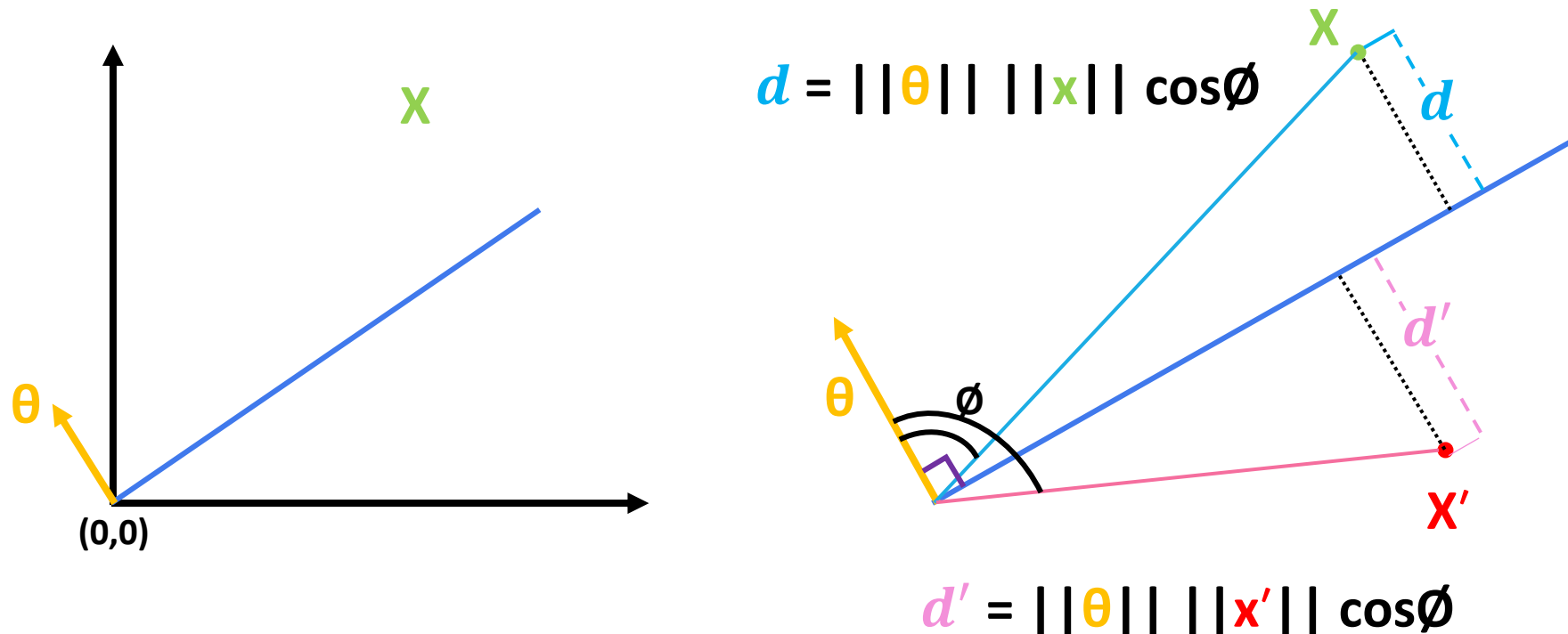
A hypothesis class H is said to **shatter** n points if there exists dataset $X=\{x^1,...,x^n\}$

- such that for any label assignment $y=(y_1,...,y_n)$ where $y_i \in \{0,1\}$, there exists a hypothesis $h \in H$ which can produce $y=h(X)$.

Learning a Class - Review

Understanding the distance of a point from the line (decision boundary)

distance of the point d = dot product of (θ, x)

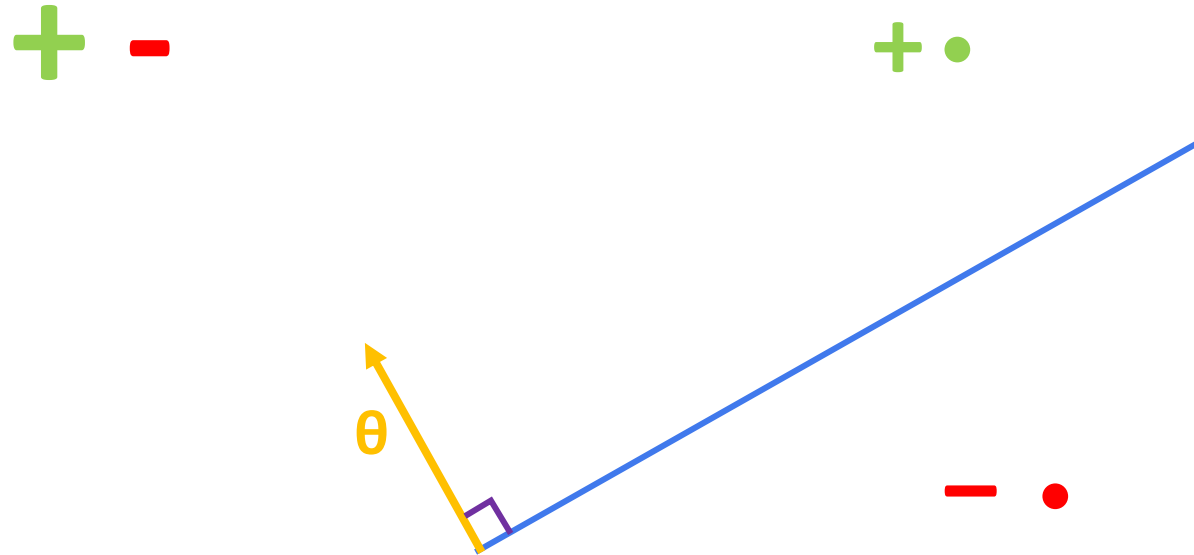


ϕ	$\cos \phi$
0	$\cos 0^\circ = 1$
30	$\cos 30^\circ = \sqrt{3}/2$
45	$\cos 45^\circ = 1/\sqrt{2}$
60	$\cos 60^\circ = 1/2$
90	$\cos 90^\circ = 0$
120	$\cos 120^\circ = -1/2$
150	$\cos 150^\circ = -\sqrt{3}/2$
180	$\cos 180^\circ = -1$
270	$\cos 270^\circ = 0$
360	$\cos 360^\circ = 1$

Shattering

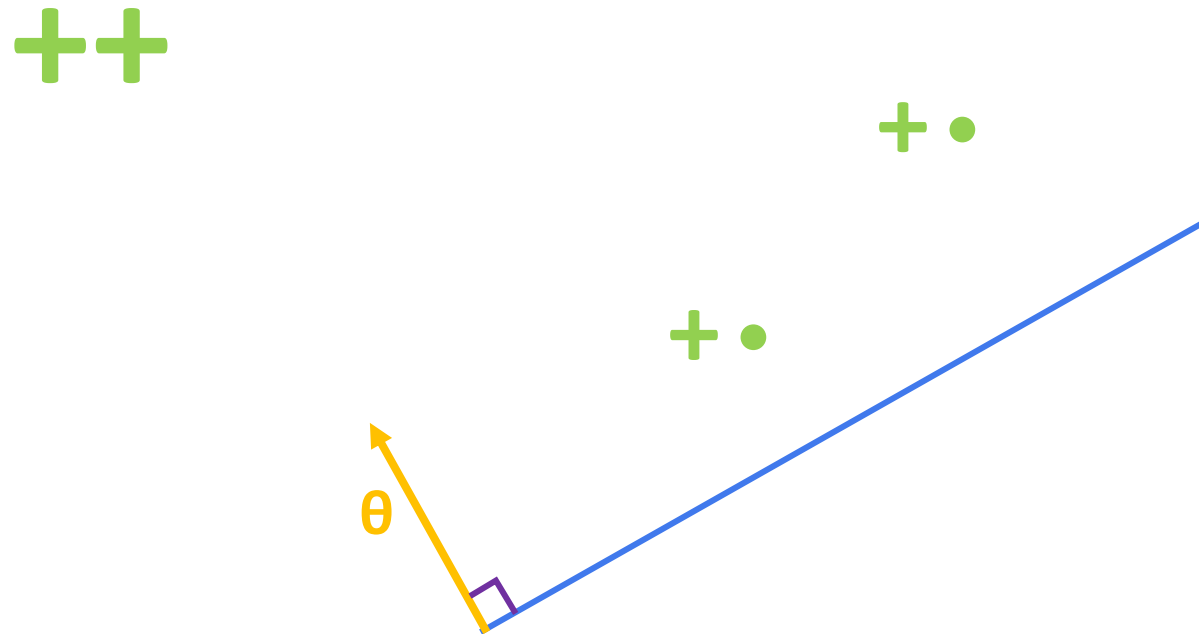
Classify (2) data points into all possible labels

$$2^2 = 4 \text{ (+-, ++, --, -+)}$$



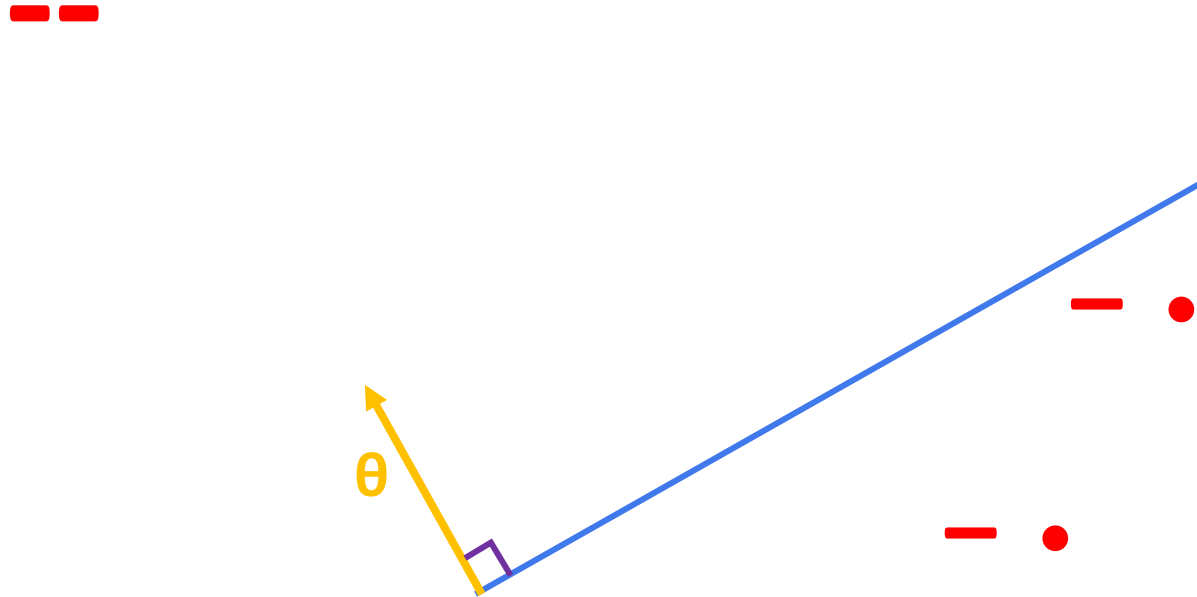
Shattering

Classify data points into all possible labels



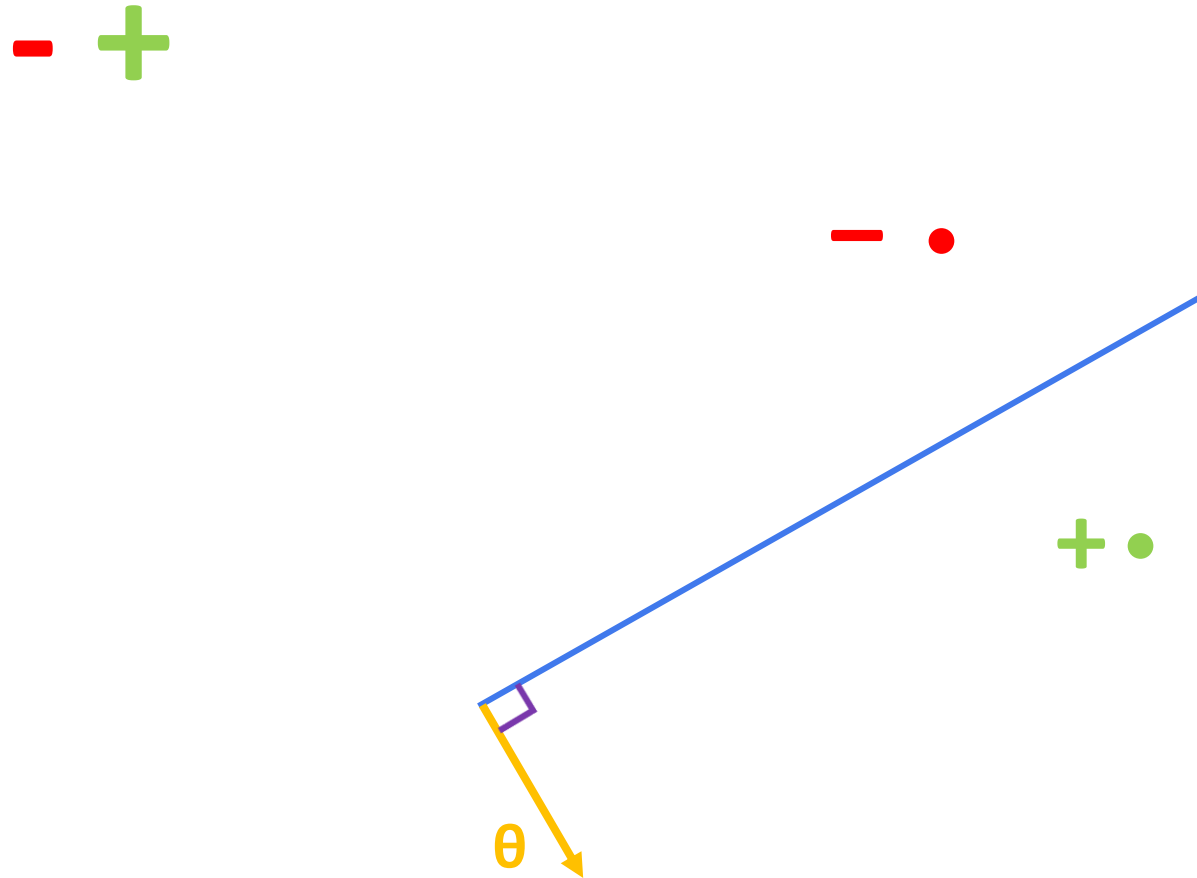
Shattering

Classify data points into all possible labels



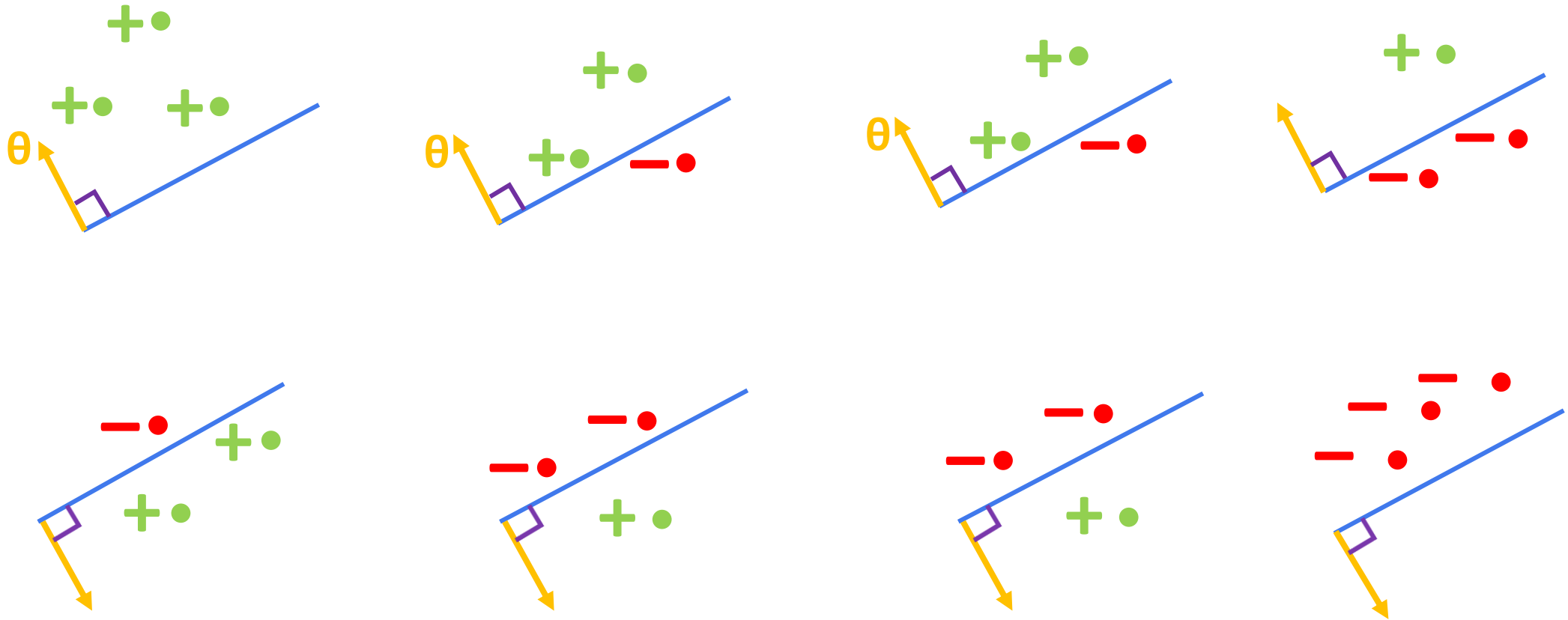
Shattering

Classify data points into all possible labels (+-, ++, --, -+)



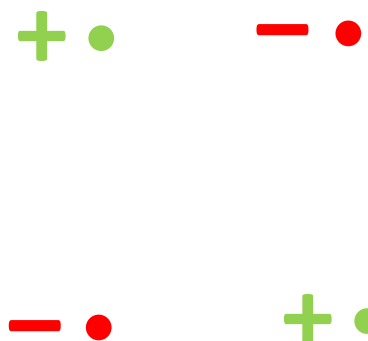
Shattering

Classify data points into all possible labels $2^3=8$ (+++, ++-, +-+, +--, -++ , --+, -+-, ---)



Shattering

Classify data points into all possible labels $2^4=16$ (++++, ++--, -+-+,, ----)



These points cannot be classified correctly with the Linear predictors. There is no linear decision boundary that separates these classes.

We have to solve this problem with non-linear strategies.

Generalization Error Bound

Generalization Error = Training Error (Empirical Risk) + Test Error (True Risk)

Generalization Error Bound:

$$L_D(h) \leq L_S(h) + \sqrt{\frac{VC(H) \left(\log \frac{2n}{VC(H)} + 1 \right) - \log \frac{\delta}{4}}{n}}$$

The term $\sqrt{\frac{VC(H) \left(\log \frac{2n}{VC(H)} + 1 \right) - \log \frac{\delta}{4}}{n}}$ is the VC dimension error bound.

VC-Dimension

- In learning theory, the VC dimension is a measure to find the **complexity of set of hypotheses H** (e.g., a set of classifiers). This notion of capacity indicates **how complicated H is**(model complexity).
- H may be able to fit well to the dataset at hand, yielding a **low training error**, there is a possibility that it overfits and gives a **high generalization error**.
- The **VC dimension** provides a tool to **analyze the generalization error** of a class of hypotheses **based on how complicated** it is.
- And this is independent of the input distribution, the target function, and the learning algorithm (i.e., a systematic approach to choose the best hypothesis $g \in H$).

VC-Dimension

- **Capacity** in VC theory is captured by **shattering**. Here we focus only on binary classification problems.
- The VC dimension can be used to **bound probabilistically the difference between the training and test errors**. This result is known as VC inequality.

Structural Risk Minimization - SRM

- We have encoded our prior knowledge by specifying a **hypothesis class H** , which we believe **includes a good predictor** for the learning task at hand.
- Yet another way to express our prior knowledge is by specifying **preferences over hypotheses within H** . That is preferring **one h from $H = \{h_1, h_2, \dots, h_n\}$**
- In the Structural Risk Minimization (SRM) paradigm, we do so by first assuming that H can be written as

$H = \bigcup_{n \in N} H_n$ and then specifying a **weight function**, $w : N \rightarrow [0, 1]$,

which assigns a **weight to each hypothesis class, H_n** , such that a **higher weight reflects a stronger preference for the hypothesis class**.

Structural Risk Minimization - SRM

- The complexity of the class of functions performing classification or regression and the algorithm's generalizability is related.
- The Vapnik-Chervonenkis (VC) theory provides a general measure of complexity and proves bounds on errors as a function of complexity.
- Structural risk minimization **is the minimization of these bounds**, which depend on the empirical risk and the capacity of the function class

Observations – Linear Predictors

- The linear Regression model is **not suitable** if the relationship between **X** and **y** is **not linear**.
- If not Linear there exists some non-linear relationship between them. For example, **y** increases **exponentially** as **X** increases.
- **Linear Regression** assumes a **normal distribution** of the response variable, which can only be applied to **continuous data**.
- If we try to build a **linear regression** model on a discrete/binary **y** variable, then the linear regression **model predicts negative values** for the corresponding response variable, which is **inappropriate**.

Observations - Linear Predictors

- We have also noticed the **Feature scaling** in Polynomial Regression. This is one of the methods of **Feature engineering**.
- When required Feature engineering on the Independent variable can be applied.
- That is instead of taking the original raw independent variables, **variable transformation** can be done, and the transformed independent variables, such as taking a log transformation, squaring the variables, and reciprocal of the variables, can also be used to build the Generalized Linear Models.
- In Logistic Regression log odds are expressed as a linear combination of the explanatory variables. Logit is the link function. The **Logistic or Sigmoid function**, returns probability as the output, which varies between 0 and 1.

Maximum Likelihood

1. Maximum likelihood is a technique to estimate parameters that have the highest probability of outputting the observed data.
2. The Maximum likelihood estimation is a method that will find the values of **Mean μ** and **Covariance σ** that result in the curve that best fits the data.
3. In maximum likelihood estimation, we want to **maximize the total probability** of the data.
4. Need to find values for θ_0 and θ_i that will yield the highest likelihood of outputting the correct true/false values.
5. The Logistic function outputs a probability y for a given value x .

Logistic Regression - Multiple Classes

The Logistic Regression model can be generalized to support multiple classes directly.

This is called **Softmax Regression**, or **Multinomial Logistic Regression**.

When given an instance $\mathbf{x}$, the Softmax Regression model first computes a score $\mathbf{s}_k(\mathbf{x})$ for each class $\mathbf{k}$,

Then estimates the probability of each class by **applying the softmax function** (also called the normalized exponential) to the scores.

The equation to compute score $\mathbf{s}_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)}$

Each class has its own dedicated parameter vector $\boldsymbol{\theta}^{(k)}$. All these vectors are stored as rows in a parameter matrix $\boldsymbol{\theta}$.

Softmax Regression

- Once the score of every class for the instance x is computed, the probability that the instance belongs to a class k is determined by applying the **Softmax function**.
- The Softmax function computes the exponential of each score, then normalizes them by dividing them with the sum of all exponentials.
- The scores are generally called Logits.
$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$
 - K is the number of classes.
 - $\mathbf{s}(\mathbf{x})$ is a vector containing the scores of each class for the instance x .
 - $\sigma(\mathbf{s}(\mathbf{x}))_k$ is the estimated probability that the instance x belongs to class k given the scores of each class for that instance.

Softmax Regression

The Softmax Regression classifier predicts the class with the highest probability

$$\hat{y} = \operatorname{argmax}_k \sigma(\mathbf{s}(\mathbf{x}))_k = \operatorname{argmax}_k s_k(\mathbf{x}) = \operatorname{argmax}_k \left(\left(\boldsymbol{\theta}^{(k)} \right)^T \mathbf{x} \right)$$

The ***argmax*** operator returns the value of a variable that maximizes a function.

In the above equation, the returns the value of k that maximizes the estimated probability

Generalized Linear Models

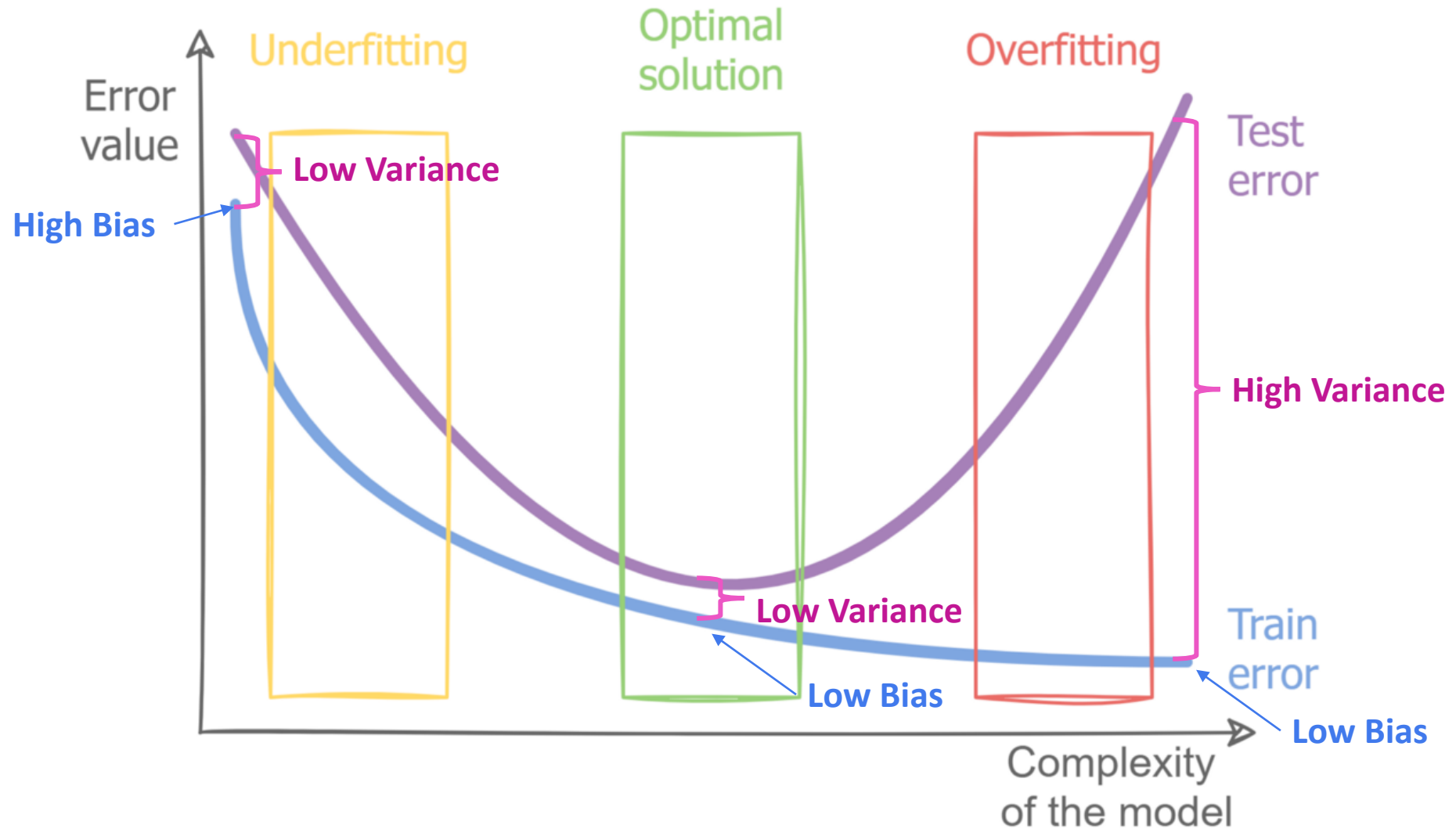
The **Generalized Linear models** will have a **low bias** for training data and **low variance** for test data.

The Generalized models produce optimal results. For a **good model**, the **train error is small** and the **val/test error is small** too.

This is possible when **training, validation, and test data** have the **same distributions**.

For Generalized Linear Models, data should **not be correlated** with each other. If the data is correlated, then the model performance will not be reliable.

Generalized Linear Models



END
